1. Войдите под пользователем user1 из практики 2 (su - user1)
2. Подсчитайте количество процессов, имеющих несколько потоков выполнения

```
root@eltex-practice2-pg1-v14:~# su user1
user1@eltex-practice2-pg1-v14:/root$ ps -eLf | awk 'NR>1 {count[$2]++} END {n=0; for (pid in count) if (count[pid] > 1) n++; print n}'
10
user1@eltex-practice2-pg1-v14:/root$
```

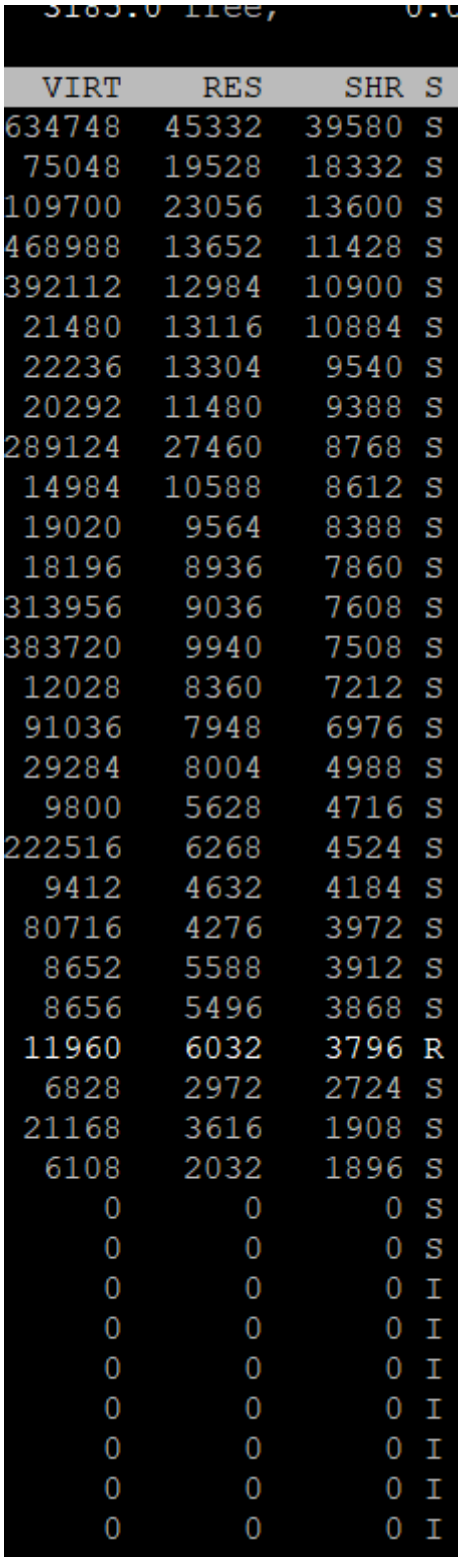
3. Запустите top и настройте вывод полей с информацией о процессе следующим образом:
   • отсортируйте список процессов по убыванию значения поля VIRT,

user1@eltex-practice2-pg1-v14: /root

```
top - 07:09:47 up 2 days,  4:25,  1 user,  load average: 0.00, 0.00, 0.00
Tasks: 108 total,   1 running, 107 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.2 us,  0.2 sy,  0.0 ni, 99.7 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   3915.9 total,   3035.3 free,    446.9 used,    661.3 buff/cache
MiB Swap:   3185.0 total,   3185.0 free,      0.0 used.   3468.9 avail Mem

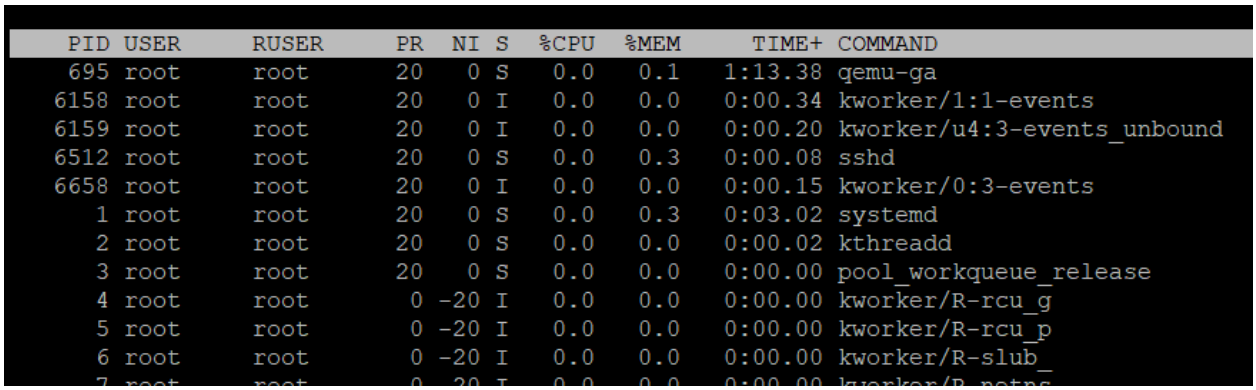
    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
    940 root      20   0  634748  45332  39580 S   0.0   1.1   0:05.82 fwupd
    702 root      20   0  468988  13652  11428 S   0.0   0.3   0:02.07 udisksd
    747 root      20   0  392112  12984  10900 S   0.0   0.3   0:00.05 ModemManager
    694 polkitd   20   0  383720   9940   7508 S   0.0   0.2   0:02.55 polkitd
    947 root      20   0  313956   9036   7608 S   0.0   0.2   0:00.02 upowerd
    366 root      rt   0  289124  27460   8768 S   0.0   0.7   0:10.46 multipathd
    720 syslog    20   0  222516   6268   4524 S   0.0   0.2   0:00.11 rsyslogd
    717 root      20   0  109700  23056  13600 S   0.0   0.6   0:00.07 unattended-upgr
    593 systemd+  20   0   91036   7948   6976 S   0.0   0.2   0:00.39 systemd-timesyn
    695 root      20   0   80716   4276   3972 S   0.0   0.1   1:13.09 qemu-ga
    316 root      19  -1   75048  19524  18332 S   0.0   0.5   0:00.78 systemd-journal
    378 root      20   0   29284   8004   4988 S   0.0   0.2   0:00.22 systemd-udevd
      1 root      20   0   22236  13304   9540 S   0.0   0.3   0:03.01 systemd
    592 systemd+  20   0   21480  13116  10884 S   0.0   0.3   0:00.23 systemd-resolve
   6518 root      20   0   21168   3616   1908 S   0.0   0.1   0:00.00 (sd-pam)
   6517 root      20   0   20292  11480   9388 S   0.0   0.3   0:00.05 systemd
    552 systemd+  20   0   19020   9564   8388 S   0.0   0.2   0:02.12 systemd-network
    700 root      20   0   18196   8936   7860 S   0.0   0.2   0:00.22 systemd-logind
   6512 root      20   0   14984  10588   8612 S   0.0   0.3   0:00.04 sshd
   6296 root      20   0   12028   8360   7212 S   0.0   0.2   0:00.00 sshd
   6683 user1     20   0   11960   6032   3796 R   0.0   0.2   0:00.02 top
    688 message+  20   0    9800   5628   4716 S   0.0   0.1   0:00.35 dbus-daemon
   6661 root      20   0    9412   4632   4184 S   0.0   0.1   0:00.00 su
   6662 user1     20   0    8656   5496   3868 S   0.0   0.1   0:00.00 bash
   6586 root      20   0    8652   5588   3912 S   0.0   0.1   0:00.01 bash
    687 root      20   0    6828   2972   2724 S   0.0   0.1   0:00.21 cron
    716 root      20   0    6108   2032   1896 S   0.0   0.1   0:00.00 agetty
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.02 kthreadd
      3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_release
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_g
      5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_p
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub_
      7 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-netns
     10 root       0 -20       0      0      0 I   0.0   0.0   0:00.28 kworker/0:0H-kblockd
     12 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-mm_pe
     13 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_kthread
```

зафиксируйте результат скриншотом, повторите для полей RES и SHR, в отчете поясните, что описывают данные показатели;



VIRT	RES	SHR	S
634748	45332	39580	S
75048	19528	18332	S
109700	23056	13600	S
468988	13652	11428	S
392112	12984	10900	S
21480	13116	10884	S
22236	13304	9540	S
20292	11480	9388	S
289124	27460	8768	S
14984	10588	8612	S
19020	9564	8388	S
18196	8936	7860	S
313956	9036	7608	S
383720	9940	7508	S
12028	8360	7212	S
91036	7948	6976	S
29284	8004	4988	S
9800	5628	4716	S
222516	6268	4524	S
9412	4632	4184	S
80716	4276	3972	S
8652	5588	3912	S
8656	5496	3868	S
11960	6032	3796	R
6828	2972	2724	S
21168	3616	1908	S
6108	2032	1896	S
0	0	0	S
0	0	0	S
0	0	0	I
0	0	0	I
0	0	0	I
0	0	0	I
0	0	0	I
0	0	0	I
0	0	0	I

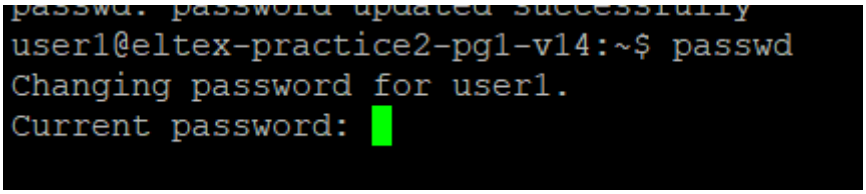
• удалите поля VIRT, RES, SHR и добавьте поле RUSER, сделайте так, чтобы это поле было показано после поля USER, поясните отличие RUSER/USER;



```
  PID USER      RUSER     PR  NI S  %CPU  %MEM     TIME+ COMMAND
  695 root      root      20   0 S   0.0   0.1   1:13.38 qemu-ga
 6158 root      root      20   0 I   0.0   0.0   0:00.34 kworker/1:1-events
 6159 root      root      20   0 I   0.0   0.0   0:00.20 kworker/u4:3-events_unbound
 6512 root      root      20   0 S   0.0   0.3   0:00.08 sshd
 6658 root      root      20   0 I   0.0   0.0   0:00.15 kworker/0:3-events
    1 root      root      20   0 S   0.0   0.3   0:03.02 systemd
    2 root      root      20   0 S   0.0   0.0   0:00.02 kthreadd
    3 root      root      20   0 S   0.0   0.0   0:00.00 pool_workqueue_release
    4 root      root       0 -20 I   0.0   0.0   0:00.00 kworker/R-rcu_g
    5 root      root       0 -20 I   0.0   0.0   0:00.00 kworker/R-rcu_p
    6 root      root       0 -20 I   0.0   0.0   0:00.00 kworker/R-slub_
```

- **VIRT** : Показывает общий объем виртуальной памяти, запрошенной процессом. На скриншоте видно, что процесс fwupd запросил больше всего виртуальной памяти (634748 KiB). Сюда входит и физическая память, и файлы, и зарезервированное пространство.
- **RES (Resident Size)**: Показывает, сколько физической оперативной памяти (RAM) процесс реально использует в данный момент. Это самый важный показатель для понимания того, сколько «реальной» памяти ест программа.
- **SHR (Shared Memory)**: Показывает объем памяти, который используется совместно с другими процессами (например, общие библиотеки).

4. В другом терминальном окне выполните команду passwd и оставьте ее в состоянии запроса текущего пароля



```
passwd: password updated successfully
user1@eltex-practice2-pg1-v14:~$ passwd
Changing password for user1.
Current password:
```

5. Перейдите в терминальное окно с top и выполните следующие действия:

• выведите все процессы, для которых реальным пользователем является пользователь, которым вы вошли в сеанс;

user1@eltex-practice2-pg1-v14: /root

```
top - 07:39:58 up 2 days,  4:55,  1 user,  load average: 0.00, 0.01, 0.00
Tasks: 114 total,   1 running, 113 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   3915.9 total,   3021.8 free,    458.4 used,    663.4 buff/cache
MiB Swap:   3185.0 total,   3185.0 free,      0.0 used.   3457.5 avail Mem

    PID USER      RUSER     PR  NI S  %CPU  %MEM     TIME+ COMMAND
   6747 user1     user1     20   0 S   0.0   0.1   0:00.00 screen
   6748 user1     user1     20   0 S   0.0   0.1   0:00.00 bash
   6763 user1     user1     20   0 S   0.0   0.1   0:00.05 screen
   6764 user1     user1     20   0 S   0.0   0.1   0:00.00 bash
   6777 user1     user1     20   0 R   0.0   0.2   0:00.19 top
   6789 user1     user1     20   0 S   0.0   0.1   0:00.00 bash
   6804 user1     user1     20   0 S   0.0   0.1   0:00.00 screen
```

• найдите процесс, запущенный командой passwd;

```
   6815 root      user1     20   0 S   0.0   0.1   0:00.00 passwd
```

• отправьте этому процессу сигналы SIGTERM, SIGINT, SIGQUIT, SIGKILL

используя их числовые идентификаторы (man 7 signal)

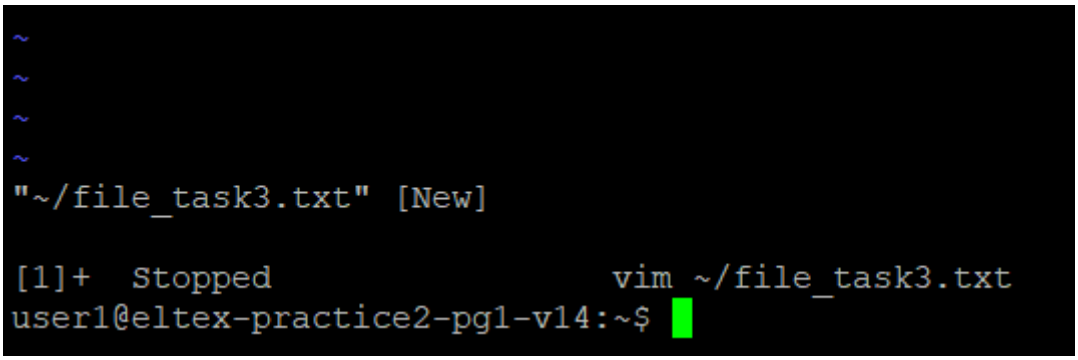
```
Send pid 6815 signal [15/sigterm] 15
    PID USER      RUSER     PR  NI S  %CPU  %MEM     TIME+ COMMAND
   6800 root      root      20   0 I   0.0   0.0   0:00.01 kworker/u4:0-events_unbound
   6808 root      root      20   0 I   0.0   0.0   0:00.00 kworker/1:2
   6809 root      root      20   0 I   0.0   0.0   0:00.00 kworker/1:3-mm_percpu_wq
   6815 root      user1     20   0 S   0.0   0.1   0:00.00 passwd
```

```
Send pid 6842 signal [15/sigterm] 2
    PID USER      RUSER     PR  NI S  %CPU  %MEM     TIME+ COMMAND
   6800 root      root      20   0 I   0.0   0.0   0:00.03 kworker/u4:0-events_unbound
   6809 root      root      20   0 I   0.0   0.0   0:00.00 kworker/1:3-mm_percpu_wq
   6822 root      root      20   0 I   0.0   0.0   0:00.00 kworker/u4:4-flush-252:0
   6831 root      root      20   0 S   0.0   0.1   0:00.00 su
   6842 root      user1     20   0 S   0.0   0.1   0:00.00 passwd
   6851 root      root      20   0 I   0.0   0.0   0:00.00 kworker/0:0
   6852 root      root      20   0 I   0.0   0.0   0:00.00 kworker/0:2
```

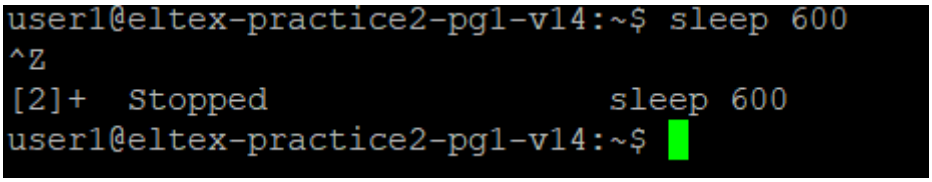
```
MiB Swap:   3185.0 total,   3185.0 free,      0.0 used.   3459.4 avail Mem
Send pid 6857 signal [15/sigterm] 3
    PID USER      RUSER     PR  NI S  %CPU  %MEM     TIME+ COMMAND
   6809 root      root      20   0 I   0.0   0.0   0:00.00 kworker/1:3-mm_percpu_wq
   6822 root      root      20   0 I   0.0   0.0   0:00.00 kworker/u4:4-flush-252:0
   6831 root      root      20   0 S   0.0   0.1   0:00.00 su
   6851 root      root      20   0 I   0.0   0.0   0:00.00 kworker/0:0
   6852 root      root      20   0 I   0.0   0.0   0:00.00 kworker/0:2
   6857 root      user1     20   0 S   0.0   0.1   0:00.00 passwd
```

```
Send pid 6871 signal [15/sigterm] 9
    PID USER      RUSER     PR  NI S  %CPU  %MEM     TIME+ COMMAND
   6809 root      root      20   0 I   0.0   0.0   0:00.00 kworker/1:3-mm_percpu_wq
   6822 root      root      20   0 I   0.0   0.0   0:00.00 kworker/u4:4-flush-252:0
   6831 root      root      20   0 S   0.0   0.1   0:00.00 su
   6851 root      root      20   0 I   0.0   0.0   0:00.00 kworker/0:0
   6852 root      root      20   0 I   0.0   0.0   0:00.00 kworker/0:2
   6871 root      user1     20   0 S   0.0   0.1   0:00.00 passwd
```

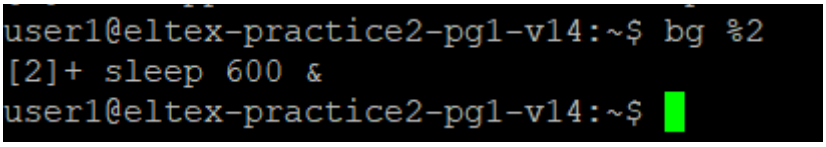
6. Выполните команду vim ~/file_task3.txt и нажмите Ctrl-Z



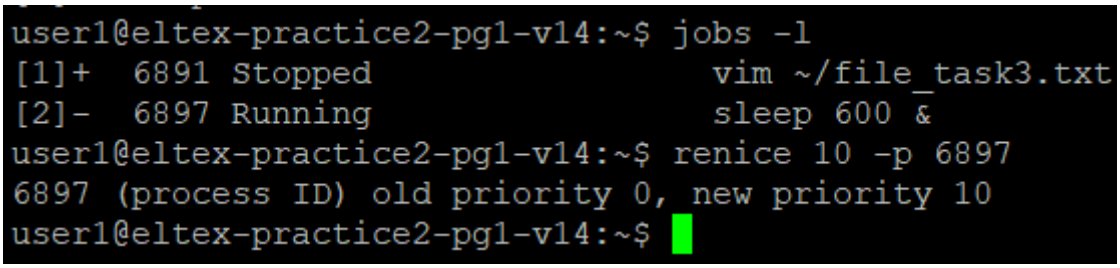
7. Выполните команду sleep 600, нажмите Ctrl-Z и выполните команду jobs



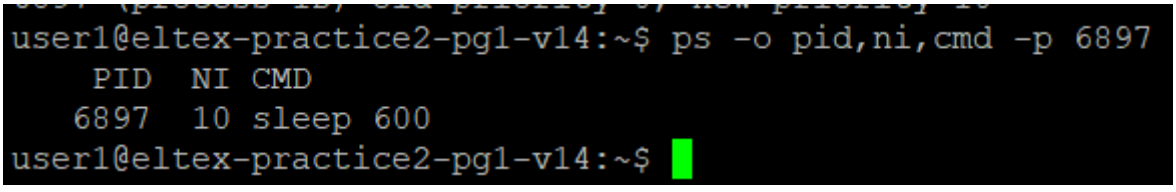
8. Последнее задание (sleep 600) сделайте фоновым



9. Измените число NICE у задания (sleep 600), сделав его равным 10

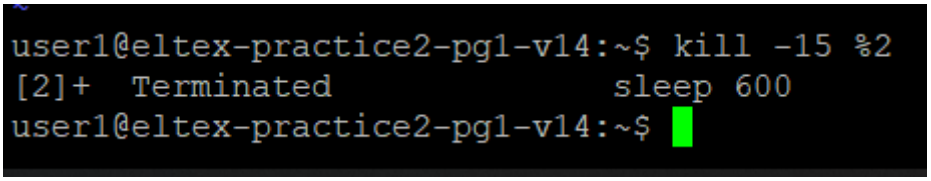


10. Проверьте, что число NICE у этого задания изменилось



11. Сделайте задание vim ~/file_ task3.txt активным и выйдите из редактора

12. Отправьте сигнал SIGTERM заданию sleep 600 и выполните команду jobs



13. Создайте перехватчик сигналов SIGINT и SIGQUIT внутри командного интерпретатора, который выводит сообщение «Меня голыми руками не возьмёшь!» (используйте встроенную команду trap) и отправьте сигналы самому себе

```
user1@eltex-practice2-pg1-v14:~$ trap 'echo "Меня голыми руками не возьмёшь!"' SIGINT SIGQUIT
user1@eltex-practice2-pg1-v14:~$ kill -SIGINT $$
Меня голыми руками не возьмёшь!
user1@eltex-practice2-pg1-v14:~$
```

14. Создайте несколько зомби-процессов несколько раз выполнив команду:

(sleep 1 & exec /bin/sleep 30) &

30 – настраиваемая задержка времени существования зомби- процесса в секундах, если не успеваете выполнить следующий пункт – увеличьте время

```
root@eltex-practice2-pg1-v14:~# (sleep 1 & exec /bin/sleep 3000) &
(sleep 1 & exec /bin/sleep 3000) &
(sleep 1 & exec /bin/sleep 3000) &
(sleep 1 & exec /bin/sleep 3000) &
[1] 6937
[2] 6939
[3] 6940
[4] 6941
root@eltex-practice2-pg1-v14:~# ps aux | grep -w Z
root        6938  0.0  0.0      0     0 pts/0    Z    08:15   0:00 [sleep] <defunct>
root        6942  0.0  0.0      0     0 pts/0    Z    08:15   0:00 [sleep] <defunct>
root        6943  0.0  0.0      0     0 pts/0    Z    08:15   0:00 [sleep] <defunct>
root        6944  0.0  0.0      0     0 pts/0    Z    08:15   0:00 [sleep] <defunct>
root        6946  0.0  0.0   6548  2388 pts/0    S+   08:15   0:00 grep --color=auto -w Z
root@eltex-practice2-pg1-v14:~#
```

15. Найдите при помощи команды ps все зомби-процессы, выведите их PID и PPID. Попробуйте досрочно завершить выполнение зомби-процессов. Все, что узнали о зомби-процессах и их особенностях в процессе выполнения пунктов опишите используя ~10 предложений в отчете.

1. Создайте скрипт на языке bash с именем cuckoo.sh, делающий следующее:

   - Создает именованный канал с именем, указанном с абсолютным путем в параметре запуска скрипта --pishitesuda=
   - Записывает в файл cuckoo.log сообщение в формате: ДАТА ВРЕМЯ Startup!
   - В процессе ожидания читает сообщения из именованного канала, на запрос «NAME[PID]: how much time do I have?" возвращает число N в диапазоне от 2 до 10, где NAME – имя обратившегося процесса и PID – его идентификатор, записывает обращение и ответ в файл cuckoo.log в формате: ДАТА ВРЕМЯ NAME[PID] N
   - При получении сигнала (на ваш выбор) скрипт записывает в файл cuckoo.log сообщение в формате: ДАТА ВРЕМЯ Shutdown! и завершает свою работу,

предварительно удалив именованный канал.

root@eltex-practice2-pg1-v14: ~

```
  GNU nano 7.2
#!/bin/bash

FIFO_PATH=""
LOG_FILE="cuckoo.log"

for arg in "$@"; do
    case $arg in
        --pishitesuda=*)
            FIFO_PATH="${arg#*=}"
            ;;
    esac
done

if [ -z "$FIFO_PATH" ]; then
    echo "Ошибка: не указан путь к именованному каналу (--pishitesuda=/path/to/fifo)
    exit 1
fi

log_message() {
    echo "$(date '+%Y-%m-%d %H:%M:%S') $1" >> "$LOG_FILE"
}

cleanup() {
    log_message "Shutdown!"
    if [ -p "$FIFO_PATH" ]; then
        rm -f "$FIFO_PATH"
    fi
    exit 0
}

trap cleanup SIGTERM SIGINT

if [ ! -p "$FIFO_PATH" ]; then
    mkfifo "$FIFO_PATH"
    chmod 666 "$FIFO_PATH"
fi

log_message "Startup!"

while true; do
    if read -r line < "$FIFO_PATH"; then
        if [[ "$line" =~ ^([^[]+)\[([0-9]+)\]:\ how\ much\ time\ do\ I\ have\?$ ]];
            NAME="${BASH_REMATCH[1]}"
            PID="${BASH_REMATCH[2]}"
```

```
root@eltex-practice2-pg1-v14:~# chmod +x /usr/local/bin/cuckoo.sh
root@eltex-practice2-pg1-v14:~#
```

2. Создайте службу systemd (unit) для запуска скрипта cuckoo.sh, разрешите его работу (enable), запустите его и проверьте состояние (status).

root@eltex-practice2-pg1-v14: ~

```
  GNU nano 7.2
[Unit]
Description=Cuckoo Daemon
After=network.target

[Service]
Type=simple
ExecStart=/usr/local/bin/cuckoo.sh --pishitesuda=/tmp/cuckoo_fifo
WorkingDirectory=/var/log
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

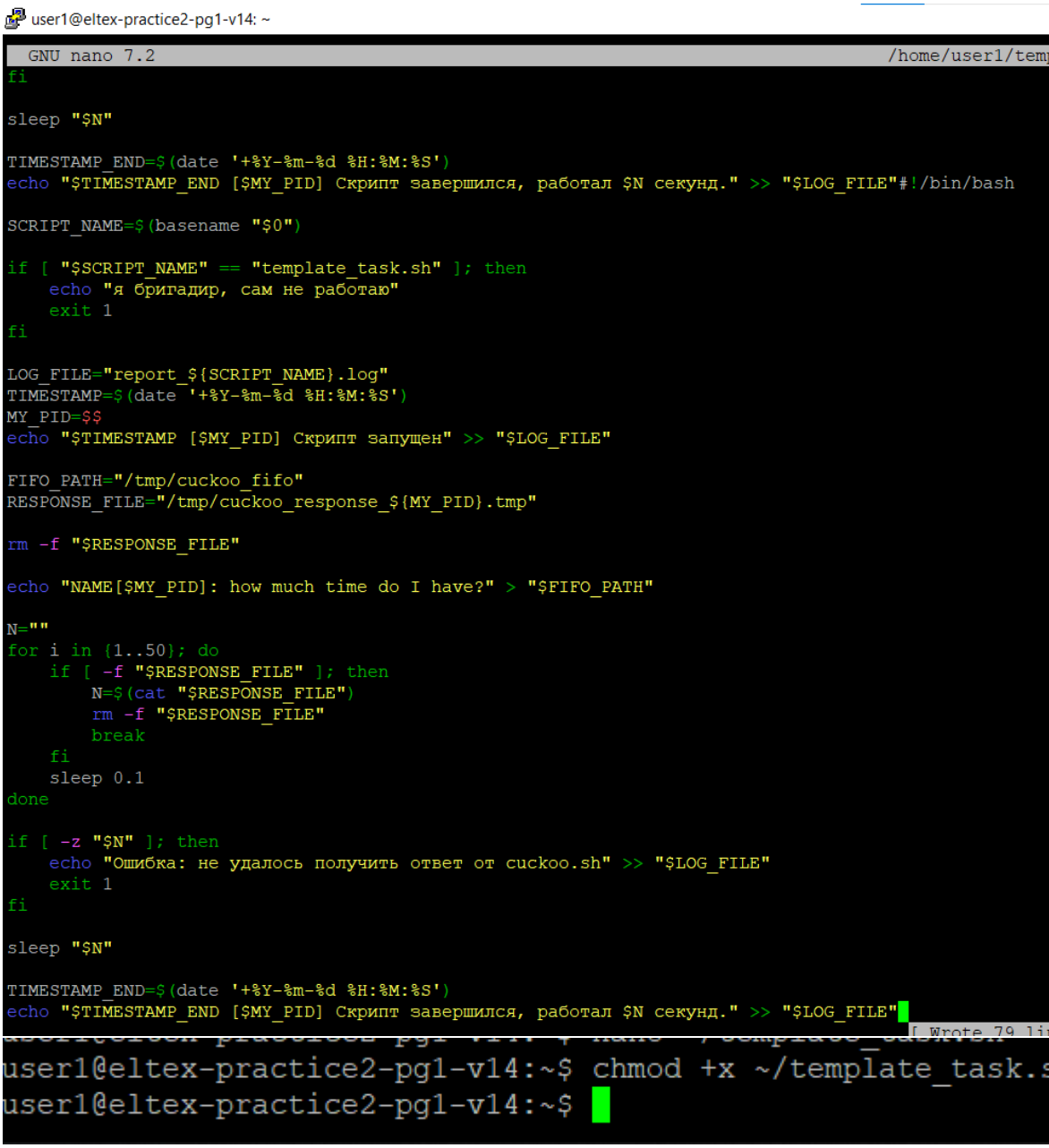
```
root@eltex-practice2-pg1-v14:~# systemctl daemon-reload
systemctl enable cuckoo.service
systemctl start cuckoo.service
systemctl status cuckoo.service
Created symlink /etc/systemd/system/multi-user.target.wants/cuckoo.service → /etc/systemd/system/cuckoo.service.
● cuckoo.service - Cuckoo Daemon
     Loaded: loaded (/etc/systemd/system/cuckoo.service; enabled; preset: enabled)
     Active: active (running) since Mon 2026-10-26 08:23:57 UTC; 104ms ago
   Main PID: 7094 (cuckoo.sh)
      Tasks: 1 (limit: 4600)
     Memory: 428.0K (peak: 908.0K)
        CPU: 3ms
     CGroup: /system.slice/cuckoo.service
             └─7094 /bin/bash /usr/local/bin/cuckoo.sh --pishitesuda=/tmp/cuckoo_fifo

Oct 26 08:23:57 eltex-practice2-pg1-v14 systemd[1]: Started cuckoo.service - Cuckoo Daemon.
root@eltex-practice2-pg1-v14:~#
```

```
root@eltex-practice2-pg1-v14:~# ls -l /tmp/cuckoo_fifo
cat /var/log/cuckoo.log
prw-rw-rw- 1 root root 0 Oct 26 08:23 /tmp/cuckoo_fifo
2026-10-26 08:23:57 Startup!
root@eltex-practice2-pg1-v14:~#
```

3. Создайте скрипт на языке bash с именем template_task.sh, делающий следующее:

   - При запуске проверят, что имя скрипта не совпадает с template_task.sh, если совпадает - выходит с уведомлением «я бригадир, сам не работаю»
   - При запуске дописывает в файл report_**имя_скрипта_без_полного_пути**.log в рабочем каталоге информацию: ДАТА ВРЕМЯ [PID] Скрипт запущен
   - Запрашивает через именованный канал скрипта cuckoo.sh, который получает из procfs, время сообщением в указанном ранее формате и ждет указанное количество секунд.
   - Дописывает в файл report_**имя_скрипта_без_полного_пути**.log сообщение: ДАТА ВРЕМЯ [PID] Скрипт завершился, работал N секунд.



4. Создайте скрипт на языке bash с именем observer.sh, читающий файл конфигурации со списком скриптов observer.conf, проверяющим их наличие в списке работающих процессов поиском в /proc и запускающих их в отключенном от терминала режиме (nohup) в случае отсутствия в нем. Информация о перезапуске дописывайте в файл observer.log

root@eltex-practice2-pg1-v14: ~

```
  GNU nano 7.2
#!/bin/bash

CONF_FILE="observer.conf"
LOG_FILE="observer.log"

if [ ! -f "$CONF_FILE" ]; then
    echo "Ошибка: файл конфигурации $CONF_FILE не найден"
    exit 1
fi

while IFS= read -r script_path || [ -n "$script_path" ]; do
    [ -z "$script_path" ] && continue
    [[ "$script_path" == \#* ]] && continue

    is_running=0
    for pid_dir in /proc/[0-9]*; do
        if [ -r "$pid_dir/cmdline" ]; then
            if tr '\0' ' ' < "$pid_dir/cmdline" | grep -q "$script_path"; then
                is_running=1
                break
            fi
        fi
    done

    if [ "$is_running" -eq 0 ]; then
        TIMESTAMP=$(date '+%Y-%m-%d %H:%M:%S')
        echo "$TIMESTAMP Запуск $script_path" >> "$LOG_FILE"
        nohup "$script_path" > /dev/null 2>&1 &
    fi
done < "$CONF_FILE"
```

5. Настройте запуск observer.sh посредством cron по расписанию – 1 раз в минуту

```
user1@eltex-practice2-pg1-v14:~$ crontab -l
* * * * * cd /home/user1 && /bin/bash /home/user1/observer.sh
user1@eltex-practice2-pg1-v14:~$
```

```
user1@eltex-practice2-pg1-v14:~$ crontab -e
crontab: installing new crontab
user1@eltex-practice2-pg1-v14:~$ crontab -l
* * * * * cd /home/user1 && /bin/bash /home/user1/observer.sh
user1@eltex-practice2-pg1-v14:~$ cat /home/user1/observer.log
ls -l /home/user1/report_*.log
2026-10-26 08:41:01 Запуск /home/user1/worker1.sh
2026-10-26 08:41:01 Запуск /home/user1/worker2.sh
2026-10-26 08:41:01 Запуск /home/user1/worker3.sh
2026-10-26 08:41:01 Запуск /home/user1/template_task.sh
-rw------- 1 user1 user1 406 Oct 26 08:30 /home/user1/report_my_worker.sh.log
user1@eltex-practice2-pg1-v14:~$
```

6. Создайте несколько символьных ссылок на файл template_task.sh с различными именами (рабочие задачи), добавьте в файл конфигурации observer.conf соответствующие записи об этих задачах, включая исходный файл template_task.sh

```
user1@eltex-practice2-pg1-v14:~$ ls -l ~/template_task.sh
-rwx------ 1 user1 user1 1939 Oct 26 08:25 /home/user1/template_task.sh
user1@eltex-practice2-pg1-v14:~$ ln -s /home/user1/template_task.sh /home/user1/worker1.sh
ln -s /home/user1/template_task.sh /home/user1/worker2.sh
ln -s /home/user1/template_task.sh /home/user1/worker3.sh
user1@eltex-practice2-pg1-v14:~$ ls -l ~/worker*.sh
lrwxrwxrwx 1 user1 user1 28 Oct 26 08:41 /home/user1/worker1.sh -> /home/user1/template_task.sh
lrwxrwxrwx 1 user1 user1 28 Oct 26 08:41 /home/user1/worker2.sh -> /home/user1/template_task.sh
lrwxrwxrwx 1 user1 user1 28 Oct 26 08:41 /home/user1/worker3.sh -> /home/user1/template_task.sh
user1@eltex-practice2-pg1-v14:~$
```

7. Соберите статистику работы в виде набора файлов report_*.log, observer.log, cuckoo.log приложите их вместе с исходными текстами скриптов в качестве отчета в виде сжатого архива tar. Не забудьте остановить процессы, удалив задачу в cron и остановив и запретив службу!